TRIGGERS

Need for assertions and triggers

- Sum of loans taken by a customer does not exceed 100,000
- Student cannot take the same course after getting a pass grade in it

Triggers

- A **trigger** is a stored procedure in database which automatically invokes whenever a special event in the database occurs

- It use to monitor a database and take a corrective action when a condition occurs

  - Examples:
    - Charge Rs.200 overdraft fee if the balance of an account after a withdrawal transaction is less than Rs.5000
    - Limit the salary increase of an employee to no more than 5% raise

Triggers

```
CREATE TRIGGER trigger-name
trigger-time trigger-event
ON table-name
FOR EACH ROW (or FOR EACH STATEMENT)
 trigger-action;
```

Triggers

The procedure performs some actions, e.g.,

- Check certain values
- Fill in some values
- Inserts/deletes/updates other records
- Check that some business constraints are satisfied
- Commit (approve the transaction) or roll back (cancel the transaction)

Trigger Components

- **Three components**
  - **Event:** When this event happens, the trigger is activated
  - **Condition (optional):** If the condition is true, the trigger executes, otherwise skipped
  - **Action:** The actions performed by the trigger

- **Semantics**
  - When the **Event** occurs and **Condition** is true, execute the **Action**

Lets see how to define these components

Trigger: Events

- **Three event types**
  - Insert
  - Update
  - Delete
- **Two triggering times**
  - Before the event
  - After the event
- **Two granularities**
  - Execute for each row
  - Execute for each statement

1) Trigger: Event

Trigger name

```
Create Trigger <name>
Before|After     Insert|Update|Delete  ON <tablename>

....
```

That is the event

- Example

```
Create Trigger ABC
Before  Insert On Students
....
```

This trigger is activated when an insert statement is issued, but before the new record is inserted

```
Create Trigger XYZ
After Update On Students
....
```

This trigger is activated when an update statement is issued and after the update is executed

Granularity of Event

- A single SQL statement may update, delete, or insert many records at the same time
  - **E.g., Update student set gpa = gpa x 0.8;**

- **Does the trigger execute for each updated or deleted record, or once for the entire statement ?**
  - **We define such granularity**

```
Create Trigger <name>
Before| After        Insert| Update| Delete

For Each Row | For Each Statement
....
```

This is the event

This is the granularity

Example: Granularity of Event

```
Create Trigger XYZ
After Update ON <tablename>

For each statement
....
```

This trigger is activated once (per UPDATE statement) after all records are updated

```
Create Trigger XYZ
Before Delete ON <tablename>

For each row
....
```

This trigger is activated before deleting each record

2) Trigger: Condition

- This component is **optional**

```
Create Trigger <name>
Before| After      Insert| Update| Delete On <tableName>
For Each Row | For Each Statement

When <condition>
…
```

That is the condition

If the employee salary > 150,000 then some actions will be taken

```
Create Trigger EmpSal
After Insert or Update On Employee
For Each Row
When (new.salary >150,000)
…
```

3) Trigger: Action

- **Action depends on what you want to do, e.g.:**
  - Check certain values
  - Fill in some values
  - Inserts/deletes/updates other records
  - Check that some business constraints are satisfied
  - Commit (approve the transaction) or roll back (cancel the transaction)

- **In the action, you may want to reference:**
  - The new values of inserted or updated records ***(:new)***
  - The old values of deleted or updated records ***(:old)***

Trigger: Referencing Values

- **In the action, you may want to reference:**
  - The new values of inserted or updated records ***(:new)***
  - The old values of deleted or updated records ***(:old)***

```
Create Trigger EmpSal
After Insert or Update On Employee
For Each Row
When (new.salary >150,000)
Begin
        if (:new.salary < 100,000) …
End;
```

Trigger body

Inside "When", the "new" and "old" should not have ":"

Inside the trigger body, they should have ":"

Trigger: Referencing Values (Cont'd)

- **Insert Event**
  - Has only :new defined
- **Delete Event**
  - Has only :old defined
- **Update Event**
  - Has both :new and :old defined
- **Before triggering (for insert/update)**
  - Can update the values in :new
  - Changing :old values does not make sense
- **After triggering**
  - Should not change :new because the event is already done

Create table

- CREATE TABLE people (age **INT**, name varchar(150));

Create Trigger

- mysql> delimiter //
- mysql> CREATE TRIGGER agecheck

```
BEFORE INSERT ON people
FOR EACH ROW
IF NEW.age < 0 THEN
SET NEW.age = 0;
END IF;
//
```

Insert data

- INSERT INTO people VALUES (-20, ‘Sid’);
- INSERT INTO people VALUES (30, ‘Josh’);

Check the value

- SELECT age FROM people;

SQL Triggers: An Example

```
mysql> select * from employee;
+----+------+---------+--------+------------+------+
| id | name | superid | salary | bdate      | dno  |
+----+------+---------+--------+------------+------+
|  1 | john |       3 | 100000 | 1960-01-01 |    1 |
|  2 | mary |       3 |  50000 | 1964-12-01 |    3 |
|  3 | bob  |    NULL |  80000 | 1974-02-07 |    3 |
|  4 | tom  |       1 |  50000 | 1970-01-17 |    2 |
|  5 | bill |    NULL |   NULL | 1985-01-20 |    1 |
+----+------+---------+--------+------------+------+
5 rows in set (0.00 sec)

mysql> select * from deptsal;
+---------+-------------+
| dnumber | totalsalary |
+---------+-------------+
|       1 |      100000 |
|       2 |       50000 |
|       3 |      130000 |
+---------+-------------+
3 rows in set (0.00 sec)
```

- We want to create a trigger to update the total salary of a department when a new employee is hired

SQL Triggers: An Example

- Create a trigger to update the total salary of a department when a new employee is hired:

```
mysql> delimiter !
mysql> create trigger update_salary
    -> after insert on employee
    -> for each row
    -> begin
    ->       if new.dno is not null then
    ->           update deptsal
    ->           set totalsalary = totalsalary + new.salary
    ->           where dnumber = new.dno;
    ->       end if;
    -> end !
Query OK, 0 rows affected (0.06 sec)

mysql> delimiter ;
```

- The keyword “new” refers to the new row inserted

SQL Triggers: An Example

```
mysql> select * from deptsal;
+---------+-------------+
| dnumber | totalsalary |
+---------+-------------+
|       1 |      100000 |
|       2 |       50000 |
|       3 |      130000 |
+---------+-------------+
3 rows in set (0.00 sec)

mysql> insert into employee values (6,'lucy',null,90000,'1981-01-01',1);
Query OK, 1 row affected (0.08 sec)

mysql> select * from deptsal;
+---------+-------------+
| dnumber | totalsalary |
+---------+-------------+
|       1 |      190000 |
|       2 |       50000 |
|       3 |      130000 |
+---------+-------------+
3 rows in set (0.00 sec)

mysql> insert into employee values (7,'george',null,45000,'1971-11-11',null);
Query OK, 1 row affected (0.02 sec)

mysql> select * from deptsal;
+---------+-------------+
| dnumber | totalsalary |
+---------+-------------+
|       1 |      190000 |
|       2 |       50000 |
|       3 |      130000 |
+---------+-------------+
3 rows in set (0.00 sec)

mysql> drop trigger update_salary;
Query OK, 0 rows affected (0.00 sec)
```

totalsalary increases by 90K

totalsalary did not change

SQL Triggers: An Example

- A trigger to update the total salary of a department when an employee tuple is modified:

```
mysql> delimiter ¦
mysql> create trigger update_salary2
    -> after update on employee
    -> for each row
    -> begin
    ->       if old.dno is not null then
    ->          update deptsal
    ->          set totalsalary = totalsalary - old.salary
    ->          where dnumber = old.dno;
    ->       end if;
    ->       if new.dno is not null then
    ->          update deptsal
    ->          set totalsalary = totalsalary + new.salary
    ->          where dnumber = new.dno;
    ->       end if;
    -> end ¦
Query OK, 0 rows affected (0.06 sec)
```

SQL Triggers: An Example

```
mysql> delimiter ;
mysql> select * from employee;
+----+--------+---------+--------+------------+------+
| id | name   | superid | salary | bdate      | dno  |
+----+--------+---------+--------+------------+------+
|  1 | john   |       3 | 100000 | 1960-01-01 |    1 |
|  2 | mary   |       3 |  50000 | 1964-12-01 |    3 |
|  3 | bob    |    NULL |  80000 | 1974-02-07 |    3 |
|  4 | tom    |       1 |  50000 | 1970-01-17 |    2 |
|  5 | bill   |    NULL |   NULL | 1985-01-20 |    1 |
|  6 | lucy   |    NULL |  90000 | 1981-01-01 |    1 |
|  7 | george |    NULL |  45000 | 1971-11-11 | NULL |
+----+--------+---------+--------+------------+------+
7 rows in set (0.00 sec)

mysql> select * from deptsal;
+---------+-------------+
| dnumber | totalsalary |
+---------+-------------+
|       1 |      190000 |
|       2 |       50000 |
|       3 |      130000 |
+---------+-------------+
3 rows in set (0.00 sec)

mysql> update employee set salary = 100000 where id = 6;
Query OK, 1 row affected (0.03 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from deptsal;
+---------+-------------+
| dnumber | totalsalary |
+---------+-------------+
|       1 |      200000 |
|       2 |       50000 |
|       3 |      130000 |
+---------+-------------+
3 rows in set (0.00 sec)
```

SQL Triggers: An Example

- A trigger to update the total salary of a department when an employee tuple is deleted:

```
mysql> delimiter !
mysql> create trigger update_salary3
    -> before delete on employee
    -> for each row
    -> begin
    ->       if (old.dno is not null) then
    ->           update deptsal
    ->           set totalsalary = totalsalary - old.salary
    ->           where dnumber = old.dno;
    ->       end if;
    -> end !
Query OK, 0 rows affected (0.08 sec)

mysql> delimiter ;
```

SQL Triggers: An Example

```
mysql> select * from employee;
+----+--------+---------+--------+------------+------+
| id | name   | superid | salary | bdate      | dno  |
+----+--------+---------+--------+------------+------+
|  1 | john   |       3 | 100000 | 1960-01-01 |    1 |
|  2 | mary   |       3 |  50000 | 1964-12-01 |    3 |
|  3 | bob    |    NULL |  80000 | 1974-02-07 |    3 |
|  4 | tom    |       1 |  50000 | 1970-01-17 |    2 |
|  5 | bill   |    NULL |   NULL | 1985-01-20 |    1 |
|  6 | lucy   |    NULL | 100000 | 1981-01-01 |    1 |
|  7 | george |    NULL |  45000 | 1971-11-11 | NULL |
+----+--------+---------+--------+------------+------+
7 rows in set (0.00 sec)
```

```
mysql> select * from deptsal;
+---------+-------------+
| dnumber | totalsalary |
+---------+-------------+
|       1 |      200000 |
|       2 |       50000 |
|       3 |      130000 |
+---------+-------------+
3 rows in set (0.00 sec)
```

```
mysql> delete from employee where id = 6;
Query OK, 1 row affected (0.02 sec)

mysql> delete from employee where id = 7;
Query OK, 1 row affected (0.03 sec)

mysql> select * from deptsal;
+---------+-------------+
| dnumber | totalsalary |
+---------+-------------+
|       1 |      100000 |
|       2 |       50000 |
|       3 |      130000 |
+---------+-------------+
3 rows in set (0.00 sec)
```

Stored Procedures in MySQL

- A stored procedure contains a sequence of SQL commands stored in the database so that it can be invoked later by a program

Stored Procedures in MySQL

Create Procedure <proc-name>

(param_spec$_1$, param_spec$_2$, …, param_spec$_n$)

Begin

-- execution code

end;

where each param_spec is of the form:

[in | out | inout] <param_name> <param_type>

- in mode: allows you to pass values into the procedure,
- out mode: allows you to pass value back from procedure to the calling program

Examples

```
CREATE PROCEDURE GetAllProducts()
BEGIN
   SELECT *  FROM products;
END
```

- Executing a stored procedure

```
CALL stored_procedure_name(argument_list);

eg: CALL GetAllProducts();
```

Examples

```
CREATE PROCEDURE GetOfficeByCountry(
    IN countryName VARCHAR(25)
)
BEGIN
    SELECT *
    FROM offices
    WHERE country = countryName;
END
```

Stored Procedures in MySQL

Example

```
CREATE PROCEDURE GetOrderCountByStatus (
    IN  orderStatus VARCHAR(25),
    OUT total INT
)
BEGIN
    SELECT COUNT(orderNumber)
    INTO total
    FROM orders
    WHERE status = orderStatus;
END
```

Stored Procedures in MySQL

Example

```
CREATE PROCEDURE SetCounter(
    INOUT counter INT,
    IN inc INT
)
BEGIN
    SET counter = counter + inc;
END
```

Example

```
mysql> select * from employee;
+----+------+---------+--------+------------+------+
| id | name | superid | salary | bdate      | dno  |
+----+------+---------+--------+------------+------+
|  1 | john |       3 | 100000 | 1960-01-01 |    1 |
|  2 | mary |       3 |  50000 | 1964-12-01 |    3 |
|  3 | bob  |    NULL |  80000 | 1974-02-07 |    3 |
|  4 | tom  |       1 |  50000 | 1978-01-17 |    2 |
|  5 | bill |    NULL |   NULL | 1985-01-20 |    1 |
+----+------+---------+--------+------------+------+
```

```
mysql> select * from department;
+---------+-------------+
| dnumber | dname       |
+---------+-------------+
|       1 | Payroll     |
|       2 | TechSupport |
|       3 | Research    |
+---------+-------------+
```

- Suppose we want to keep track of the total salaries of employees working for each department

```
mysql> create table deptsal as
    -> select dnumber, 0 as totalsalary from department;
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> select * from deptsal;
+---------+-------------+
| dnumber | totalsalary |
+---------+-------------+
|       1 |           0 |
|       2 |           0 |
|       3 |           0 |
+---------+-------------+
```

ocedure to

Example

```
mysql> delimiter //
```

Step 1: Change the delimiter (i.e., terminating character) of SQL statement from semicolon (;) to something else (e.g., //)
So that you can distinguish between the semicolon of the SQL statements in the procedure and the terminating character of the procedure definition

Example

```
mysql> delimiter //
mysql> create procedure updateSalary (IN param1 int)
    -> begin
    ->     update deptsal
    ->     set totalsalary = (select sum(salary) from employee where dno = param1)
    ->     where dnumber = param1;
    -> end; //
Query OK, 0 rows affected (0.01 sec)
```

Step 2:

1. Define a procedure called updateSalary which takes as input a department number.
2. The body of the procedure is an SQL command to update the totalsalary column of the deptsal table.
3. Terminate the procedure definition using the delimiter you had defined in step 1 (//)

Example

```
mysql> delimiter //
mysql> create procedure updateSalary (IN param1 int)
    -> begin
    ->     update deptsal
    ->     set totalsalary = (select sum(salary) from employee where dno = param1)
    ->     where dnumber = param1;
    -> end; //
Query OK, 0 rows affected (0.01 sec)

mysql> delimiter ;
```

Step 3: Change the delimiter back to semicolon (;)

Example

```
mysql> call updateSalary(1);
Query OK, 0 rows affected (0.00 sec)

mysql> call updateSalary(2);
Query OK, 1 row affected (0.00 sec)

mysql> call updateSalary(3);
Query OK, 1 row affected (0.00 sec)
```

Step 4: Call the procedure to update the totalsalary for each department

Example

```
mysql> select * from deptsal;
+---------+-------------+
| dnumber | totalsalary |
+---------+-------------+
|       1 |      100000 |
|       2 |       50000 |
|       3 |      130000 |
+---------+-------------+
3 rows in set (0.00 sec)
```

Step 5: Show the updated total salary in the deptsal table

Stored Procedures in MySQL

- Use show procedure status to display the list of stored procedures you have created

```
mysql> show procedure status;
+------+---------------+-----------+---------+---------------------+---------------------+----------
-----+---------+----------------------+----------------------+--------------------+
| Db   | Name          | Type      | Definer | Modified            | Created             | Security_
type | Comment | character_set_client | collation_connection | Database Collation |
+------+---------------+-----------+---------+---------------------+---------------------+----------
-----+---------+----------------------+----------------------+--------------------+
| ptan | updateSalary0 | PROCEDURE | ptan@%  | 2010-03-16 12:21:55 | 2010-03-16 12:21:55 | DEFINER
     |         | latin1               | latin1_swedish_ci    | latin1_swedish_ci  |
+------+---------------+-----------+---------+---------------------+---------------------+----------
-----+---------+----------------------+----------------------+--------------------+
1 row in set (0.02 sec)
```

- Use drop procedure to remove a stored procedure

```
mysql> drop procedure updateSalary;
Query OK, 0 rows affected (0.00 sec)
```

Stored Procedures in MySQL

- You can declare variables in stored procedures
- You can use flow control statements (conditional IF-THEN-ELSE or loops such as WHILE and REPEAT)
- MySQL also supports cursors in stored procedures.
  - A cursor is used to iterate through a set of rows returned by a query so that we can process each individual row.
- To learn more about stored procedures, go to: http://www.mysqltutorial.org/mysql-stored-procedure-tutorial.aspx

Example using Cursors

- The previous procedure updates one row in deptsal table based on input parameter
- Suppose we want to update all the rows in deptsal simultaneously
  - Fir

```
mysql> update deptsal set totalsalary = 0;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 3  Changed: 0  Warnings: 0

mysql> select * from deptsal;
+---------+-------------+
| dnumber | totalsalary |
+---------+-------------+
|       1 |           0 |
|       2 |           0 |
|       3 |           0 |
+---------+-------------+
3 rows in set (0.00 sec)
```

Example using Cursors

```
mysql> delimiter $$
mysql> drop procedure if exists updateSalary$$
Query OK, 0 rows affected (0.00 sec)

mysql> create procedure updateSalary()
    -> begin
    ->         declare done int default 0;
    ->         declare current_dnum int;
    ->         declare dnumcur cursor for select dnumber from deptsal;
    ->         declare continue handler for not found set done = 1;
    ->
    ->         open dnumcur;
    ->
    ->         repeat
    ->               fetch dnumcur into current_dnum;
    ->               update deptsal
    ->               set totalsalary = (select sum(salary) from employee
    ->                                  where dno = current_dnum)
    ->               where dnumber = current_dnum;
    ->         until done
    ->         end repeat;
    ->
    ->         close dnumcur;
    -> end$$
Query OK, 0 rows affected (0.00 sec)

mysql> delimiter ;
```

Drop the old procedure

Use cursor to iterate the rows

Example using Cursors

- Call procedure

```
mysql> select * from deptsal;
+---------+-------------+
| dnumber | totalsalary |
+---------+-------------+
|       1 |           0 |
|       2 |           0 |
|       3 |           0 |
+---------+-------------+
3 rows in set (0.01 sec)

mysql> call updateSalary;
Query OK, 0 rows affected (0.00 sec)

mysql> select * from deptsal;
+---------+-------------+
| dnumber | totalsalary |
+---------+-------------+
|       1 |      100000 |
|       2 |       50000 |
|       3 |      130000 |
+---------+-------------+
3 rows in set (0.00 sec)
```

Another Example

- Create a procedure to give a raise to all employees

```
mysql> select * from emp;
+----+--------+---------+--------+------------+------+
| id | name   | superid | salary | bdate      | dno  |
+----+--------+---------+--------+------------+------+
|  1 | john   |       3 | 100000 | 1960-01-01 |    1 |
|  2 | mary   |       3 |  50000 | 1964-12-01 |    3 |
|  3 | bob    |    NULL |  80000 | 1974-02-07 |    3 |
|  4 | tom    |       1 |  50000 | 1978-01-17 |    2 |
|  5 | bill   |    NULL |   NULL | 1985-01-20 |    1 |
|  6 | lucy   |    NULL |  90000 | 1981-01-01 |    1 |
|  7 | george |    NULL |  45000 | 1971-11-11 | NULL |
+----+--------+---------+--------+------------+------+
7 rows in set (0.00 sec)
```

Another Example

```
mysql> delimiter |
mysql> create procedure giveRaise (in amount double)
    -> begin
    ->        declare done int default 0;
    ->        declare eid int;
    ->        declare sal int;
    ->        declare emprec cursor for select id, salary from employee;
    ->        declare continue handler for not found set done = 1;
    ->
    ->        open emprec;
    ->        repeat
    ->               fetch emprec into eid, sal;
    ->               update employee
    ->               set salary = sal + round(sal * amount)
    ->               where id = eid;
    ->        until done
    ->        end repeat;
    -> end |
Query OK, 0 rows affected (0.00 sec)
```